Malware Analysis Report

RAT.Unknown.exe

Malware Analyst: Giovanni Ocasio

Contents

Executive Summary 3
Static Analysis 4
- **Hashes** 4
- **Data Size** 4
- **Strings** 4
- **Windows APIs** 5

Dynamic Analysis 6
- **Initial Detonation** 6
- **Network-Based Signatures** 7
  - **WireShark** 7
- **Host-Based Signatures** 8
  - **ProcMon** 8
  - **TCPView** 9
  - **Correlation** 11

Executive Summary

Analysis of the malware identified as *RAT.Unknown.exe* reveals that it initiates an HTTP request to a remote EC2 instance (http://serv1.ec2-102-95-13-2-ubuntu.local), likely in an attempt to retrieve a secondary payload. Upon successful download, the malware proceeds to open TCP port 5555, establishing a bind shell on the infected host. This configuration enables the malware author to remotely connect and execute commands with the same privileges as the user who executed the malware.

To mitigate this threat, it is recommended to:

- Blacklist the associated EC2 instance by both hostname and public IP address.
- Report the malicious use of the EC2 instance to Amazon Web Services (AWS) for further investigation and takedown.
- Monitor for the creation of and connections to uncommon or unauthorized ports, particularly TCP port 5555, across the network.

Static Analysis

Hashes

MD5	689FF2C6F94E31ABBA1DDEBF68BE810E
SHA1	69B8ECF6B7CDE185DAED76D66100B6A31FD1A668
SHA256	248D491F89A10EC3289EC4CA448B19384464329C442BAC395F680C4F3A345C8C

Data Size

Virtual Size	87,384
Raw Data Size	87,552
Assumption: Malware is unpacked	

Strings

CLSID_XMLHTTPRequest/ IID_IXMLHttpRequest

@[+] what command can I run for you

@Connection: Keep-Alive/ @Connection/ @Host: / @ HTTP/1.1/ @User-Agent/ @user-agent

InternetOpenW/ InternetOpenUrlW/ @wininet

MessageBoxW/ @NO SOUP FOR YOU/ @\mscordll.exe/ @Nim httpclient/1.0.6/ @/msdcorelib.exe

@AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup

@intrt explr

@http://serv1.ec2-102-95-13-2-ubuntu.local

Windows APIs

Enumeration	
GetCurrentProcess	Used to retrieve a handle for the current process.
GetProcessId	Used to retrieve the process identifier of the specified process.
GetCurrentThreadId	Used to retrieve the thread identifier of the calling thread.
VirtualQuery	Used to retrieve information about a range of pages within the virtual address space of a specified process. This function is commonly used by malware to enumerate an external process.

Injection	
VirtualAlloc	Used by malware to allocate memory as part of process injection.
VirtualProtect	Used by malware to modify memory protection (often to allow write or execution).
CreateProcess	Used to create a process. This function is used by malware in several process injection attacks such as process hollowing.

Helpers	
CreatePipe	Used to create an anonymous pipe and returns handles to the read and write ends of the pipe. Could be used to add a sub-process for execution via cmd.
CreateNamedPipe	**NO DATA**
WriteFile	Used to write data to the specified file or input/output (I/O) device.

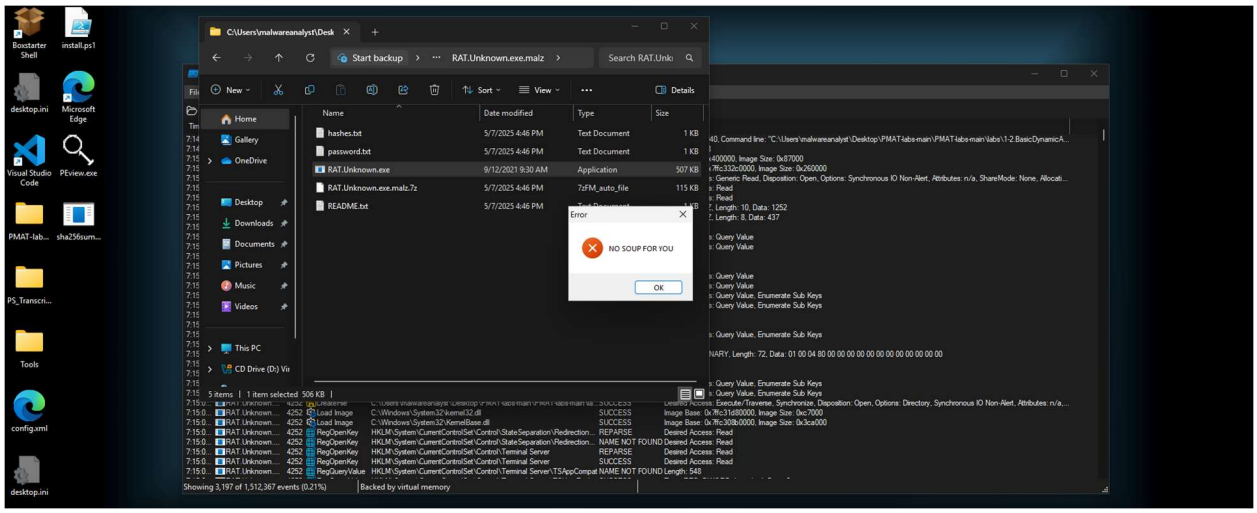
Anti-Debugging	
QueryPerformanceFrequency	Used to retrieve the frequency of the performance counter. This function is commonly used by malware for anti-debugging purposes. The malware will measure the time before and after an operation, if the time exceeds taken expected time, the malware will terminate or activate a benign function.

Internet	
InternetOpen	Used to initialize the use of WinINet functions
InternetOpenUrl	Used to open a resource specified by a complete FTP or HTTP URL.

Unknown	
RtlAddFunctionTable	**NO DATA**

Dynamic Analysis

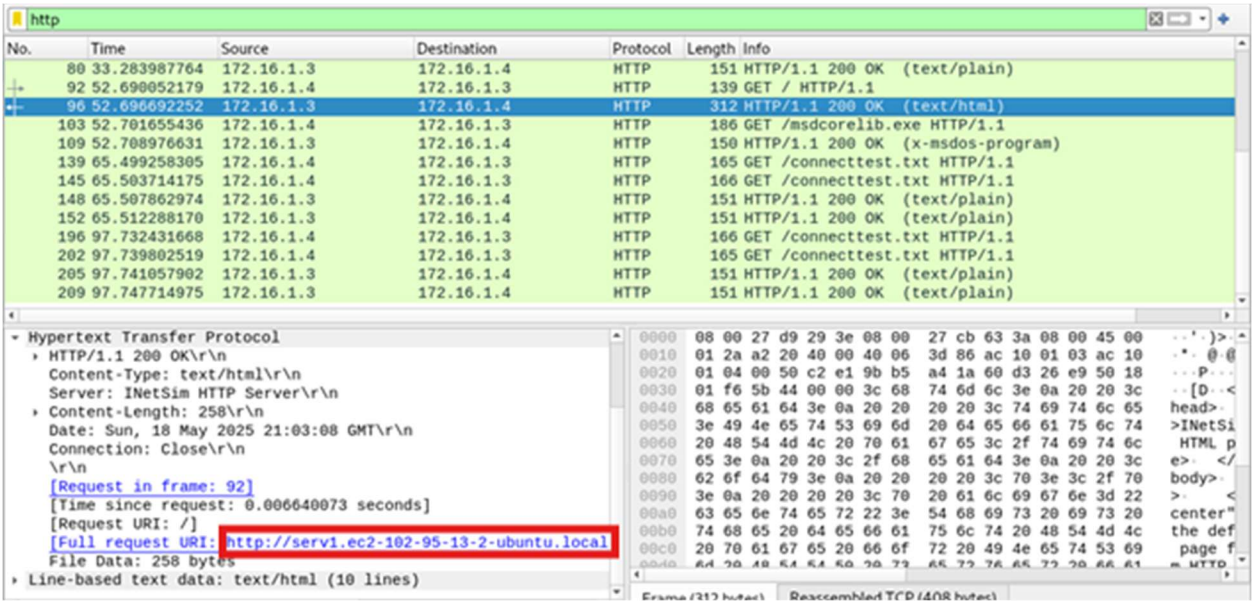
Initial Detonation



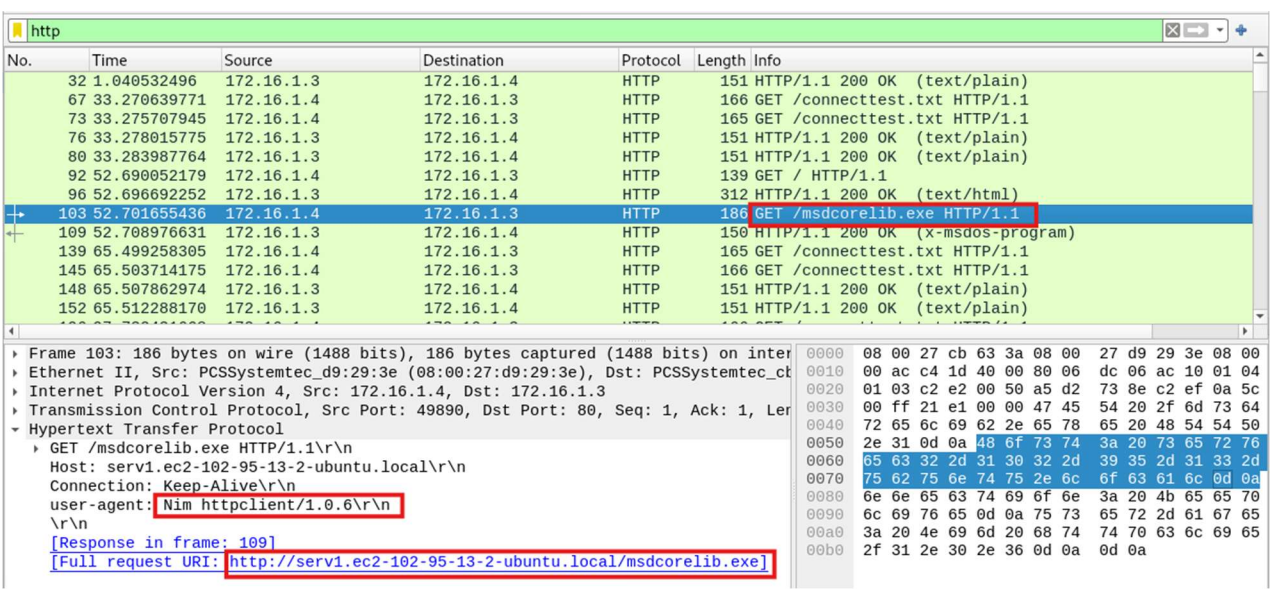
Network-Based Signatures

WireShark

When running the malware with WireShark enabled we can see that the malware makes an HTTP request to an EC2 instances:



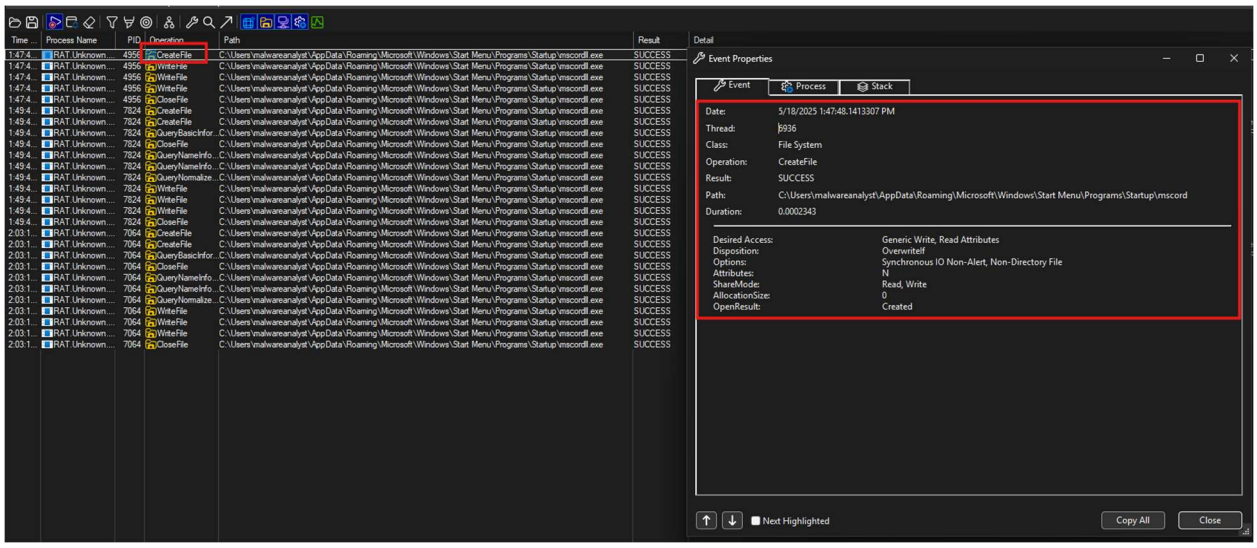
The malware then sends a GET request for *msdcorelib.exe*. Note it uses a unique User-Agent:



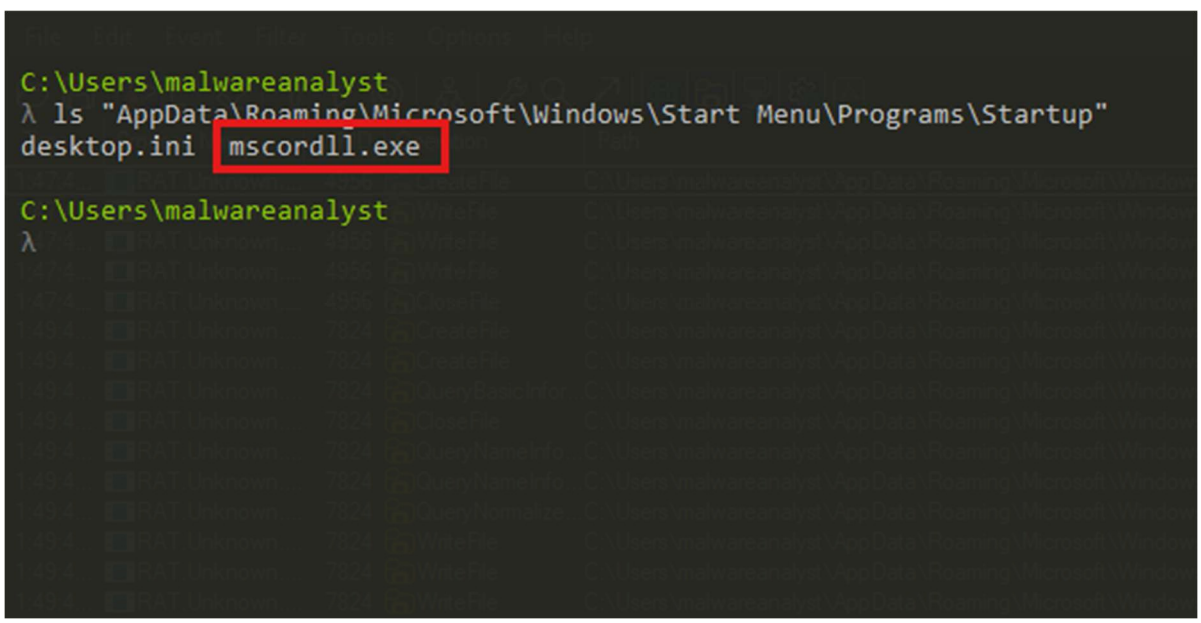
Host-Based Signatures

ProcMon

Monitoring the malware's detonation on ProcMon reveals that it wrote a file to a directory that was specified in the binaries code. (Review Strings):

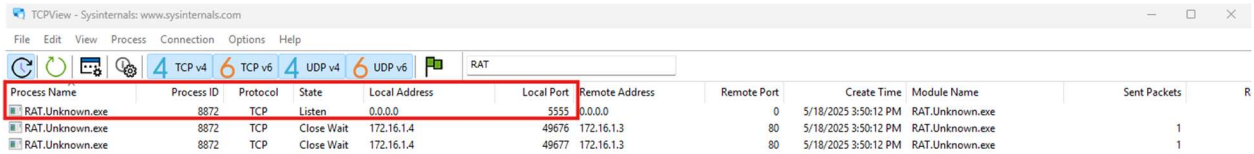


We can see that the malware did, in fact write to this directory. This would cause the *msdcordll.exe* binary that it installed to execute every time this user logins to this device:

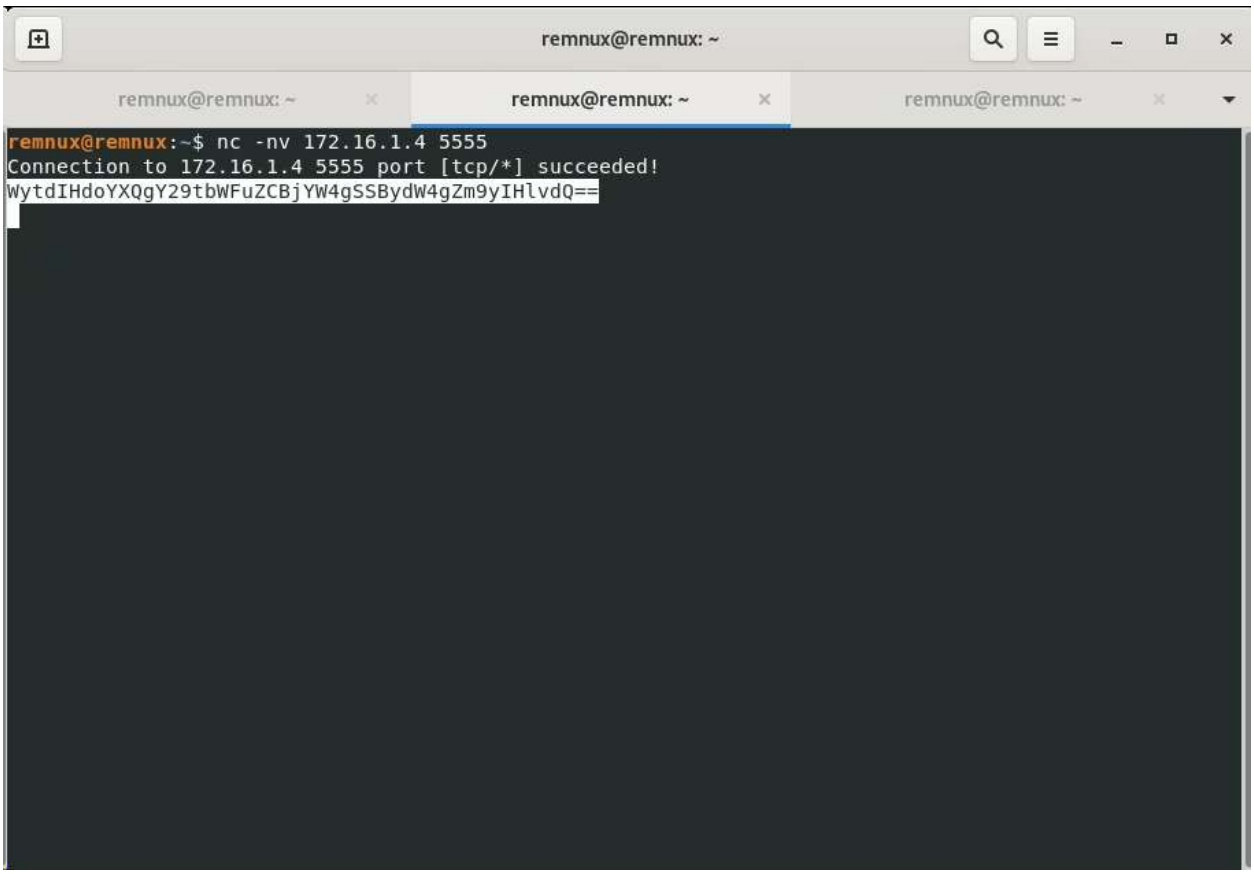


TCPView

When monitoring the interactions with the host on TCPView, the can analyst has confirmed that the malware begins listening on port 5555:



Connecting to this open socket provides a Base64 encoded message. When this message is decoded, it outputs a string found in the source code of the malware binary (Review Strings section):



```
remnux@remnux:~$ echo "WvtdIHdoYXQgY29tbWFuZCBjYW4gSSBydW4gZm9yIHlvdQ==" | base64 -d
[+] what command can I run for youremnux@remnux:~$
```

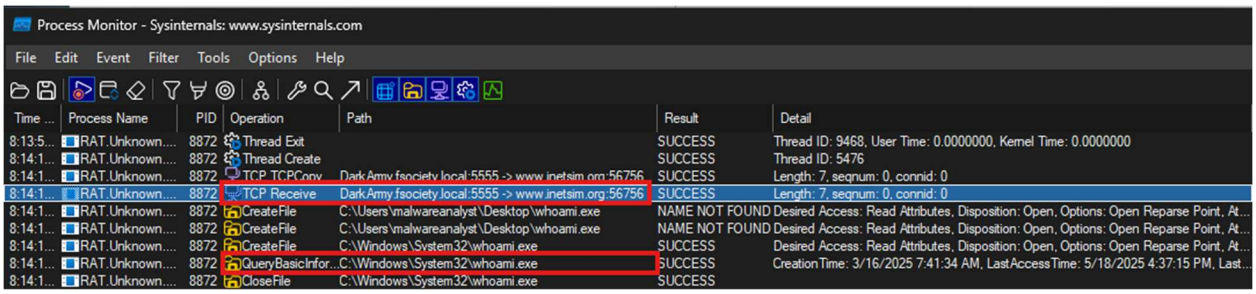
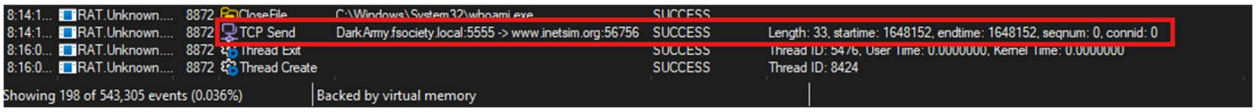
Based off this output, the analyst can assume that this bind shell can execute system commands. Testing this reveals a base64 output, which when decoded reveals that the shell does run system commands:

```
remnux@remnux: ~
remnux@remnux: ~   remnux@remnux: ~   remnux@remnux: ~
remnux@remnux:~$ nc -nv 172.16.1.4 5555
Connection to 172.16.1.4 5555 port [tcp/*] succeeded!
WvtdIHdoYXQgY29tbWFuZCBjYW4gSSBydW4gZm9yIHlvdQ==
whoami
ZGFya2FybXlcbWFsd2FyZWFuYWx5c3QK
```

```
remnux@remnux: ~
remnux@remnux: ~   remnux@remnux: ~   remnux@remnux: ~
remnux@remnux:~$ echo "ZGFya2FybXlcbWFsd2FyZWFuYWx5c3QK" | base64 -d
darkarmy\malwareanalyst
remnux@remnux:~$
```

Correlation

With the information gathered from TCPView, the analyst can go back to ProcMon to correlate the data and view the TCP connection:



In ProcMon, the TCP request is received over the bind port 5555 and the whoami.exe executable is run. After which, the data is then sent over TCP sockets on port 5555 to the remote host: